

Révision MLOps — Fiche complète (20 séances)

Cours : Orchestration Machine Learning (ESGI / IABD). Fil rouge du cours = churn ;
ton projet = détection de fraude bancaire (même pipeline, autre dataset).

Le concept central

MLOps = DevOps appliqué au Machine Learning. Industrialiser le cycle de vie d'un modèle : **versionner, tester, déployer, surveiller**. Un modèle utile est un modèle **déployé et surveillé**, pas un notebook.

4 piliers : Données (versionnées/validées), Entraînement (reproductible/tracé), Déploiement (CI/CD), Surveillance (drift/performance).

Cycle de vie : Concevoir → Industrialiser → Exploiter (boucle).

MODULE 1 — Fondations

S1 — Introduction au MLOps

- Notebook = exécution manuelle, état caché, non reproductible, jetable.
- Pipeline = automatisé, reproductible, testé, versionné, code de production.
- 4 piliers : Données, Entraînement, Déploiement, Surveillance.
- À retenir : un modèle utile est déployé + surveillé ; le MLOps industrialise tout le cycle.

S2 — Environnement & outils

- « Ça marche chez moi » n'est pas une garantie → **un venv par projet**.
- `pip install -e "[dev]"` = mode editable (importable + modifiable). `PYTHONPATH=src`.
- Outils : **Git** (versionner), **venv** (isoler), **ruff** (lint/format), **mypy** (types).
- Un **script sh** rejoue la mise en place ; un **Makefile** centralise (`install`, `lint`, `type`, `test`, `check`).
- À retenir : `make check` = lint + types + tests d'un coup.

S3 — Baseline du fil rouge

- Une **baseline** = modèle simple de référence (régression logistique) pour mesurer les gains.
 - **Split stratifié** (`stratify=y`, `random_state=42`) préserve le taux de classes.
 - Preprocessing : **ColumnTransformer** = `StandardScaler` (num) + `OneHotEncoder` (cat).
 - Métriques : **F1 & ROC AUC** priment sur l'**accuracy** (trompeuse si déséquilibré).
 - Artefact : `models/model.joblib`.
-

MODULE 2 — Entraînement

S4 — Reproductibilité & validation

- Un seul split peut mentir → **validation croisée k-fold** (`StratifiedKFold`) : moyenne + écart-type.
- **Underfitting** (trop simple, biais) vs **Overfitting** (trop complexe, variance).
- **Fuite de données** : le preprocessing se `fit` sur le **train seul** → utiliser un **Pipeline**. Jamais de fit sur le test.
- Repro = **seed + données figées + environnement épinglé**.

S5 — MLflow Tracking

- 4 composants MLflow : **Tracking, Models, Registry, Projects**.
- Vocabulaire : **Experiment** (groupe de runs) > **Run** (1 entraînement daté) > **Params / Metrics / Artifacts**.
- Architecture : Client → **Backend store** (params/métriques, SQLite/Postgres) + **Artifact store** (modèles, disque/S3).
- Logging **manuel** (`log_params / log_metrics`) vs **autolog()**.
- Registry : **None** → **Staging** → **Production**.

S6 — Optuna & Model Registry

- **Optuna** = **optimisation bayésienne (TPE)** : apprend des essais passés (mieux que grid search).
- Étude : espace de recherche → fonction **objective** (renvoie un score CV) → `study.optimize(n_trials), direction="maximize"`.
- `registered_model_name` crée/incrémente une **version** dans le Registry.

S7 — AutoML & SHAP

- Comparer plusieurs familles : **linéaire (LogReg)** / **arbres (RF)** / **boosting (XGBoost, LightGBM)**, garder le meilleur sur une métrique CV.
- **SHAP** = explicabilité : contribution de chaque feature.
- **Global** : `summary_plot` (vue d'ensemble).
- **Local** : `force_plot` (une prédiction).
- L'interprétabilité est une exigence métier.

MODULE 3 — Conteneurisation & qualité

S8 — Docker

- Le conteneur **fige code + deps + environnement** dans une **image**.
- **VM** (émule un OS complet, lourd) vs **Conteneur** (partage le noyau de l'hôte, léger).
- **Dockerfile** (recette) → **Image** (figée) → **Conteneur** (exécution) → **Registry**.
- **Cache de layers** : copier `pyproject.toml` avant le code → l'install des deps reste en cache. Ordre = du plus stable au plus volatil.
- Bonnes pratiques : image **-slim**, `.dockerignore`, versions épinglées, multi-stage.

S9 — Conteneuriser l'entraînement

- « Build once, run anywhere » : image figée = même entraînement partout (CI, serveur, cloud).
- `docker build -f ... -t ...` puis `docker run -v $(pwd)/models:/app/models` (le **volume** récupère `model.joblib`).
- L'ordre des couches conditionne le cache ; `.dockerignore` allège le contexte.

S10 — Tests pytest

- Tests = **filet de sécurité** de la CI. Structure **Arrange / Act / Assert**.
- Boîte à outils : `assert`, **fixture** (données partagées), **parametrize** (cas variés), `raises` (tester les erreurs).
- Un test vérifie une **propriété**, pas une valeur fragile. `pytest -q` avant chaque commit.

S11 — Tests données & modèle

- « Garbage in, garbage out » : un pipeline vert peut servir de mauvaises données.
- **Tests de données** = contrats : **Schéma, Plages, Nulls, Distribution**.
- **Tests de modèle** = seuil de performance (`assert f1 >= 0.60`) + comportement attendu.

- QCM intermédiaire couvre les modules 1 à 3.

MODULE 4 — Déploiement API

S12 — FastAPI

- Une API transforme un `.joblib` en **service HTTP** ; elle découple le modèle de ses clients.
- Atouts FastAPI : **async/rapide (ASGI)**, **pydantic** (validation auto), **doc auto /docs**, typé.
- **pydantic** rejette une entrée invalide en **422** sans code.
- Le modèle se charge **une fois au démarrage** (`lifespan`). Erreurs : 422 (invalide), 503 (modèle non chargé).

S13 — API reliée au Registry

- Charger `models:/nom/Production` au lieu d'un chemin en dur → **promouvoir une version** suffit à changer la prod, sans redéployer l'API.
- `/health` rend l'API surveillable.

S14 — docker-compose

- Décrit **toute la stack en un fichier YAML**, lancée d'une commande.
- Les services se joignent **par leur nom** (`http://mlflow:5000`).
- **depends_on** ordonne le démarrage ; **volumes** = persistance (sinon perte au `down`).
- `docker compose up -d --build, down` .

S15 — Tester l'API

- Tester le **contrat** (codes HTTP, schéma), pas juste le modèle.
- **TestClient** = teste l'app **sans lancer de serveur** → rapide, fiable en CI.
- **200** (nominal), **422** (invalide).

MODULE 5 — Orchestration & CI/CD

S16 — Concepts d'orchestration

- **Cron planifie mais ne surveille rien** (pas de dépendances/reprises/logs centralisés).
- **DAG** = Directed Acyclic Graph : **orienté** (dépendances) + **acyclique** (pas de boucle).
- **Task** = unité de travail ; **Run** = exécution datée.
- Propriétés : **Idempotence, Reprise, Observabilité, Planification**.
- Outils : **Airflow** (standard), Prefect, Kubeflow, GitHub Actions.

S17 — Apache Airflow

- Architecture : **Scheduler** (déclenche), **Webserver** (UI), **Executor** (workers), **Metadata DB** (état des runs).
- DAG en code : `schedule="0 3 * * 1"`, `catchup=False`, `retries` . Ordre : `a >> b >> c` .
- **XCom** transporte de petites valeurs entre tâches.
- Idempotence + garde-fous = pipeline fiable.

S18 — GitHub Actions : CI

- **CI** valide chaque push automatiquement → garder `main` vert.
- Anatomie : **Workflow** (YAML) = **Events** (push/PR/schedule) + **Jobs** (runner) + **Steps**.
- Quality gate : `ruff` + `mypy` + `pytest` . `needs` ordonne les jobs. Artefact = fichier produit (ex. `model.joblib`).
- Bonnes pratiques : cache, fail fast, **branche protégée** (CI verte avant merge).

S19 — GitHub Actions : CD

- **CD** livre quand la CI est verte : **build** → **push image (registre)** → **deploy**.
- `needs: test` enchaîne la CD après la CI.
- **Secrets** dans **GitHub Secrets** (jamais dans le code/logs).
- Stratégies : **Tag** (versionner), **Rollback**, **Env** (staging/prod), **Approval**.

S20 — Déploiement AWS & soutenance

- Le cloud apporte **disponibilité et passage à l'échelle**.
- Chaîne : **ECR** (héberge l'image) → **App Runner** (conteneur continu, API REST, auto-scaling) ou **Lambda** (à la demande, sporadique) → **URL publique**.

GLOSSAIRE

- **MLOps** : DevOps appliqué au ML (versionner, tester, déployer, surveiller).
- **Baseline** : modèle simple de référence à battre.
- **Pipeline (sklearn)** : preprocessing + modèle dans un objet ; évite la fuite de données.
- **Split stratifié** : découpe train/test en préservant le taux de classes.
- **Validation croisée (k-fold)** : k plis, entraîner sur k-1, tester sur 1, moyenner.
- **Underfitting / Overfitting** : trop simple (biais) / trop complexe (variance).
- **Fuite de données (leakage)** : info du test qui contamine l'entraînement.
- **Accuracy / F1 / ROC AUC** : F1 et ROC AUC robustes au déséquilibre.
- **MLflow Tracking** : journalise params, métriques, artefacts.
- **Experiment / Run / Params / Metrics / Artifacts** : vocabulaire MLflow.
- **Model Registry** : versionne les modèles ; stages None/Staging/Production.
- **Optuna / TPE** : optimisation bayésienne d'hyperparamètres.
- **Grid search** : recherche exhaustive sur une grille.
- **SHAP** : explicabilité (contribution des features), global/local.
- **Docker / Dockerfile / Image / Conteneur / Registry** : conteneurisation.
- **Layer / cache** : couches du Dockerfile mises en cache.
- **Volume** : stockage persistant/partagé.
- **docker-compose** : orchestration multi-services en YAML.
- **pytest / fixture / parametrize** : tests.
- **FastAPI / pydantic / uvicorn** : API, validation, serveur ASGI.
- **422 / 503** : entrée invalide / modèle non chargé.
- **TestClient** : teste l'API sans serveur.
- **Orchestration / DAG / Task / Run** : pipeline et planification.
- **Idempotence** : rejouer = même résultat.
- **Airflow / Scheduler / Webserver / Executor / Metadata DB / XCom** : orchestrateur.
- **CI / CD** : intégration continue (tests) / livraison continue (build+push image).
- **Workflow / Event / Job / Step / Runner / Action / needs** : GitHub Actions.
- **Artefact** : fichier produit par un job, téléchargeable.
- **GitHub Secrets / GHCR** : secrets / registre d'images GitHub.
- **ECR / App Runner / Lambda** : services AWS.

QCM BLANC (réponses en fin de ligne)

1. MLOps = ? → **DevOps appliqué au ML**.
2. Les 4 piliers ? → **Données, Entraînement, Déploiement, Surveillance**.
3. `pip install -e` ? → installe en mode **éditable**.
4. `stratify=y` ? → préserve le **taux de classes**.
5. Pourquoi pas l'accuracy seule ? → **trompeuse si déséquilibre** (→ F1/ROC AUC).
6. Validation croisée ? → estime la **vraie performance** (moyenne + écart-type).
7. Fuite de données → évitée par ? → un **Pipeline** (fit sur train seul).
8. Composants MLflow ? → **Tracking, Models, Registry, Projects**.

9. Stages du Registry ? → **None** → **Staging** → **Production**.
10. Optuna utilise ? → **optimisation bayésienne (TPE)**.
11. SHAP global vs local ? → `summary_plot` / `force_plot`.
12. VM vs conteneur ? → conteneur partage le **noyau**, léger.
13. Pourquoi copier pyproject avant le code (Docker) ? → **cache de layers**.
14. Volume Docker ? → **persister/partager** des données.
15. Structure d'un test ? → **Arrange / Act / Assert**.
16. Test de données vérifie ? → des **contrats** (schéma, plages, nulls, distribution).
17. Code si entrée invalide / modèle non chargé ? → **422 / 503**.
18. lifespan FastAPI ? → charge le modèle **une fois au démarrage**.
19. `models:/nom/Production` ? → **découple** API et modèle.
20. TestClient ? → teste l'API **sans serveur**.
21. Cron vs orchestrateur ? → l'orchestrateur gère **dépendances/reprises/logs**.
22. DAG ? → graphe **orienté acyclique**.
23. Idempotence ? → rejouer = **même résultat**.
24. Composants Airflow ? → Scheduler, Webserver, Executor, Metadata DB.
25. XCom ? → transporte des **valeurs entre tâches**.
26. CI vs CD ? → CI valide (tests) / CD livre (build+push image).
27. `needs` ? → **dépendance** entre jobs.
28. Où vivent les secrets ? → **GitHub Secrets** (jamais dans le code).
29. Artefact CI ? → **fichier produit** par un job, téléchargeable.
30. App Runner vs Lambda ? → conteneur **continu** / fonction **à la demande**.